Google ADK and Vertex AI Agent Runtime

A complete solution-architecture study guide — fundamentals, advanced design, and real-world use cases
Updated: 6 November 2026
Implementation language: Python (Google ADK); cloud target: Google Cloud
Learning format: Problem → concepts → minimal working solution → production evolution → trade-offs

> **Terminology (important):** Google's current managed agent hosting documentation calls the product **Agent Runtime on Gemini Enterprise Agent Platform**. Earlier material calls it **Vertex AI Agent Engine**. The Python package and resources still contain names such as `vertexai.agent_engines`, `AdkApp`, and `reasoningEngine`. In this guide, **managed Agent Runtime** refers to this Google Cloud hosting service, not the local ADK `Runner`. See official sources [G1], [G8], and [G9].
>
> **Source note:** The Google-specific API facts are grounded in the linked Google documentation. Architecture recommendations, synthetic use cases, and failure patterns are explanatory design guidance, not statements of a Google service guarantee. Code is designed for study; the local ADK sample matches documented APIs but has not been executed against your Google Cloud account. Check versions and region/model availability before deployment.

How to use this guide

Start with **Chapters 1–6** to understand and build a working agent. Use **Chapters 7–12** to study architecture and complex workflows. Use **Chapters 13–19** for managed Agent Runtime, enterprise operations, security, and deployment. Finally, study the **use-case blueprints**, **labs**, and **decision checklists**. You do not need a Google Cloud project until the managed-deployment chapter; the initial agent can use a Gemini API key.

Track	Chapters	Outcome
Foundation	1–6	Explain ADK, implement tools, and understand runner/session/event behavior
Architecture	7–12	Design orchestration, state, MCP, approvals, RAG, and interoperability
Cloud runtime	13–16	Distinguish local runtime from managed hosting; deploy and govern an agent
Production	17–19	Secure, observe, evaluate, scale, and recover agent workloads
Applied design	20–23	Compare use cases, complete labs, and document architecture decisions

Part I — Foundation

1. Begin with an actual business problem

Scenario: IT incident investigation assistant. An operator asks: “Investigate incident INC-42. Show what changed, the service health, and your evidence. Do not perform remediation.” The ticket system owns incident records. Monitoring owns live service health. The approved runbook repository owns operational procedures. None of these facts should be invented by the LLM.

Actors and responsibilities

Actor/component	Responsibility and source of truth
Operator	Authenticated user requesting investigation; may later approve a proposed action
Application backend	Validates identity, scopes a request, enforces allowed tool use, tracks run IDs
ADK agent	Chooses among permitted read-only tools and summarizes evidence
Ticket API	Definitive incident status, owner, scope, timestamps
Monitoring API	Definitive measurements and alert events
Runbook repository	Approved operational instructions and versions

Ranked design constraints: (1) access control and evidence integrity; (2) bounded tool calls and no side effects; (3) clear failure/unknown states; (4) acceptable latency; (5) controlled token and tool cost. **Out of scope initially:** autonomous changes, multi-agent orchestration, MCP servers, cloud deployment, vector databases, and browser automation.

Minimal workflow: retrieve one incident; retrieve health for its known service; summarize supported findings. If both reads are deterministic, ordinary application code might be sufficient. ADK earns its place when the operator asks flexible questions that require choosing permitted tools or combining findings. This is a **custom case** aligned with the project's orchestration teaching module.



Good response shape (illustrative, not a specific SDK wire format):

```
{
  "incident_id": "INC-42",
  "status": "investigating",
  "hypothesis": "A recent deployment may correlate with errors",
  "evidence_ids": ["incident:INC-42", "health:payments-api:2026-11-06T12:00Z"],
  "confidence_note": "Correlation is not proof of root cause",
  "proposed_action": null
}
```

Critical boundary: an agent recommendation is not an approved change. The business service must validate the operator and action independently before any real-world write.

---

2. ADK: what it is and what it is not

Google Agent Development Kit (ADK) is a code-first framework for building AI agents, tool integrations, structured workflows, and multi-agent systems. It supports local development and multiple deployment targets. ADK is **not** itself a language model, a business database, or an authorization system [G1].

Term	Plain meaning	Example
Model	Generates text/structured output and may propose tool calls	Gemini proposes `get_incident("INC-42")`
Agent	Model plus instructions, tools, and execution behavior	Incident investigator
Tool	Callable capability with typed arguments and results	`get_incident(incident_id)`
Runner	Executes an agent for a turn and produces events	Local Python runner
Session	Conversation and associated state for a user	Repeated incident questions
State	Typed/shared values needed by the workflow	Incident ID, evidence IDs, approval status
Memory	Information retrieved or retained across conversations where authorized	Approved user preference, long-term notes

Term	Plain meaning	Example
Artifact	Stored non-chat output	CSV result, report, document
Workflow	Explicit ordered/branched orchestration	Retrieve → validate → summarize
Managed Agent Runtime	Cloud service that hosts deployed agent code	Agent Platform endpoint

Separate the software layers

```
+-----------------------------------------------------+
| User interface / custom web service                 |
+-----------------------------------------------------+
                         |
                         v
+-----------------------------------------------------+
| ADK application and execution                        |
| Agent / Runner / workflow / tools / sessions         |
+-----------------------------------------------------+
          |                  |                 |
          v                  v                 v
+----------------+ +----------------+ +----------------+
| Model endpoint | | Business APIs  | | Storage/Memory |
+----------------+ +----------------+ +----------------+

Managed deployment option:
+-----------------------------------------------------+
| Google Cloud Agent Runtime hosts agent application   |
| Infrastructure and platform services are managed    |
+-----------------------------------------------------+
```

Important: an ADK agent can run locally, on Cloud Run, on GKE, or on the managed Agent Runtime. **ADK framework ≠ Vertex AI model inference ≠ managed Agent Runtime hosting.** Hosting does not automatically authorize a tool to retrieve private customer records [G1], [G8], [G10].

---

3. Install ADK and create a project

Official ADK Python quickstart [G2] describes the following workflow. Use an isolated Python environment, and avoid committing `.env` files or API keys.

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install --upgrade google-adk
adk create incident_agent
```

Expected project layout:

```
project_root/
  incident_agent/
    __init__.py
    agent.py
    .env                # local secrets; never commit
```

For the Gemini API-key quickstart, set `GOOGLE_API_KEY` in the project `.env` or an appropriate secret source. **Do not hardcode an API key in `agent.py`.** Use a currently supported Gemini model; the official local quickstart uses `gemini-flash-latest` as an example. Model names and availability change, so verify them in the Google model catalog [G2].

```
# From the parent directory containing incident_agent/
adk run incident_agent
# Or for local debugging only:
adk web --port 8000
```

`adk web` provides a **development UI**, not a production customer-facing application [G2]. For production, build your own user-facing service or use a supported managed integration.

---

4. First working read-only ADK agent

Replace `incident_agent/agent.py` with the following self-contained mock-tool example. Only the Gemini model call requires credentials/network; the two tool functions use synthetic data and can be tested in ordinary Python.

```
from google.adk.agents import Agent

INCIDENTS = {
    "INC-42": {
        "incident_id": "INC-42",
        "service": "payments-api",
        "status": "investigating",
        "summary": "Elevated 5xx errors after a deployment",
        "evidence_id": "incident:INC-42",
    }
}

HEALTH = {
    "payments-api": {
        "service": "payments-api",
        "status": "degraded",
        "error_rate_pct": 7.2,
        "evidence_id": "health:payments-api:sample-1",
    }
}


def get_incident(incident_id: str) -> dict:
    """Look up one synthetic incident by its exact ID (read-only)."""
    item = INCIDENTS.get(incident_id)
    if item is None:
        return {"status": "not_found", "incident_id": incident_id}
    return {"status": "success", "incident": item}


def get_service_health(service: str) -> dict:
    """Return a synthetic service-health snapshot (read-only)."""
    item = HEALTH.get(service)
    if item is None:
        return {"status": "not_found", "service": service}
    return {"status": "success", "health": item}


root_agent = Agent(
    name="incident_agent",
    model="gemini-flash-latest",  # confirm model availability
    description="Read-only incident investigation assistant",
    instruction=(
        "You investigate incidents using approved read-only tools. "
        "Call get_incident for an incident ID. "
        "Use only returned service names when calling get_service_health. "
        "State evidence IDs in your answer. "
        "Do not invent facts or claim root cause without evidence. "
        "Never propose that you executed remediation. "
        "If evidence is missing, say what you could not verify."
    ),
    tools=[get_incident, get_service_health],
)
```

Run `adk run incident_agent`, then ask:

> Investigate INC-42. Report its status and latest health evidence. Do not change anything.

Expected behavior: the agent selects the incident tool, may use its returned service name to call the health tool, and explains the result with evidence identifiers. **The exact wording and number/order of model calls are not deterministic.**

Negative cases to test: incident `INC-999` returns `not_found`; an unsupported service returns `not_found`; a user's request to delete a service has no matching tool and therefore cannot be executed by these tools. A prompt alone is not authorization—future write tools must enforce policy separately.

---

5. How an ADK request actually runs

A beginner often sees only “prompt in, answer out.” Architecture requires understanding the intermediate events and ownership.

```
User request
    |
    v
Application authenticates caller
    |
    v
Runner retrieves/creates session
    |
    v
Agent receives instruction + context + allowed tools
    |
    v
Model suggests tool call --------------+
    |                                  |
    v                                  |
Tool dispatcher validates and invokes  |
    |                                  |
    v                                  |
Tool result becomes an event -----------+
    |
    v
Model synthesizes response
    |
    v
Runner emits final event / persists session
```

The **Runner** orchestrates the agent execution lifecycle. The **SessionService** manages conversation records/state. **Events** represent output, model/tool activity, and state changes. A session ID identifies a conversation but must not be treated as authorization. ADK's runtime event-loop and session documentation explain these components [G3], [G4].

A commonly documented programmatic runner pattern uses `Runner`, `InMemorySessionService`, and `google.genai.types`. For durable applications, replace the in-memory service with a supported persistent service appropriate to the deployment. Refer to the exact SDK-version example before integrating this into a server.

```
# Illustrative Runner integration pattern — check against pinned ADK version.
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types
from incident_agent.agent import root_agent

APP_NAME = "incident_study"
USER_ID = "mock-operator"
SESSION_ID = "local-session-1"

session_service = InMemorySessionService()
runner = Runner(
    agent=root_agent,
    app_name=APP_NAME,
    session_service=session_service,
)

# Create a session before running (asynchronous):
# await session_service.create_session(
#     app_name=APP_NAME, user_id=USER_ID, session_id=SESSION_ID
# )
# user_message = types.Content(
#     role="user", parts=[types.Part(text="Investigate INC-42")]
# )
# async for event in runner.run_async(
#     user_id=USER_ID, session_id=SESSION_ID,
#     new_message=user_message,
# ):
#     print(event)
```

Design rule: do not assume a tool ran successfully just because text says so. Use actual tool results, events, or authoritative business records as evidence.

---

6. Sessions, state, memory, artifacts, and context

These five ideas solve different problems. Treating them as one storage bucket causes privacy leaks and unreliable workflows.

Concept	Scope	What belongs there	What does not belong there
Session	Conversation timeline	Turns, event history, session metadata	Permanent business source of truth
State	Working values for steps/ turns	Approved workflow stage, evidence references	Unlimited raw documents or unreviewed secrets
Memory	Authorized longer-lived knowledge	Reviewed preferences or useful history	Arbitrary private facts without consent/ retention policy
Artifact	Larger typed/binary output	Investigation report, CSV, generated file	A substitute for business approval or database record
Model context	Tokens sent to model this turn	Relevant instructions, tool data, summarized history	Entire enterprise dataset

ADK documents scoped state prefixes such as `user:`, `app:`, and `temp:`. **Temporary invocation state is not durable business state.** ADK also exposes session/memory/artifact services and context-compression features [G4], [G5].

Example state contract (application-owned proposal, not official ADK wire schema):

```
{
  "session_id": "sess-42",
  "incident_id": "INC-42",
  "workflow_status": "investigating",
  "evidence_ids": ["incident:INC-42"],
  "pending_action_id": null,
  "last_tool_status": "success"
}
```

Security requirements: derive user/tenant from verified server identity; keep state scoped by tenant and user; expire it according to policy; never trust `user_id` or `tenant_id` supplied by model-generated text. Memory should support correction and deletion. A search index is a derived copy of data, not the source of truth.

Part II — Architecture and orchestration

7. Tool architecture: the most important ADK boundary

Agents become useful when they access real capabilities. The tool boundary is where untrusted model suggestions meet business systems.

Four common integration choices

Approach	When it fits	What to watch
Python function tool	Small app, direct API wrapper, simple local development	Model input validation, auth, timeout
MCP toolset	Reusable tools shared across AI clients	Server trust, transport, credentials, version support
OpenAPI-based tools	Existing well-described REST API	Schema quality, server-side authorization
Agent as tool/sub-agent	Specialist reasoning within a controlled agent system	Limits, state sharing, cost, ambiguous ownership

Google ADK documents function tools, `McpToolset`, OpenAPI tools, authentication integration, and tool confirmations [G6].

Tool contract proposal:

```
{
  "name": "get_incident",
  "input": {"incident_id": "INC-42"},
  "output": {
    "status": "success",
    "incident": {"incident_id": "INC-42", "service": "payments-api"}
  },
  "errors": ["not_found", "forbidden", "timeout", "upstream_error"]
}
```

Strong design patterns: use typed inputs; constrain IDs and enum values; return explicit error status; bound duration/response size; log a correlation ID; reauthorize each record/tool call; write only through an auditable service. A `description` or tool schema helps the model choose a tool, but it does not grant access to data.

MCP with ADK

ADK's documented Python MCP integration uses `McpToolset` and connection parameters such as `StreamableHTTPConnectionParams` for a remote MCP server [G6]. A conceptual structure is:

```
# Framework shape based on Google's MCP integration documentation.
# Supply a real, authorized MCP server URL and authentication strategy.
import os
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import (
    StreamableHTTPConnectionParams,
)

mcp_tools = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url=os.environ["REMOTE_MCP_URL"],
        timeout=5,
        sse_read_timeout=300,
    )
)
# Then include mcp_tools in the agent's tools list.
```

For multi-user enterprise traffic, dynamic per-user identity forwarding and server-side policy matter more than a hardcoded service token. Google documents header providers and confirmation options for the toolset; verify current API and auth compatibility before implementing [G6a].

MCP is not required if one agent calls one private Python function. Introduce it when independent clients benefit from a reusable protocol boundary.

---

8. Agent orchestration patterns

There are two separate questions: **who decides the next step?** and **who owns durable progress?** An LLM can suggest an action; application/workflow code should retain control of requirements and permissions.

Pattern	What decides routing?	Appropriate scenario	Main drawback
Single LLM agent with tools	Model within supplied tool set	Flexible read-only Q&A	May over-call tools or misroute
Sequential workflow	Code-specified order	Extract → validate → report	Less adaptive
Parallel workflow	Code-specified fan-out	Independent document analyses	Costs and merge conflicts
Loop workflow	Code-specified repetition/exit	Bounded revisions	Runaway loop without caps
Supervisor/specialists	Parent agent or code routing	Distinct specialist expertise	More latency and tracing complexity
Graph workflow (ADK 2.0)	Explicit nodes/edges/ branches	Typed business workflow	More initial modeling
A2A delegation	External service contract	Independent remote agent	Remote identity and task failure

Google documents `SequentialAgent`, `ParallelAgent`, and `LoopAgent` as template workflows, plus **ADK 2.0 graph workflows**. The official sequential-workflow page says graph and dynamic workflows are more flexible starting with ADK 2.0 for supported languages [G7], [G7a].

Example: deterministic sequential pipeline

```
from google.adk.agents import LlmAgent, SequentialAgent

MODEL = "gemini-flash-latest"  # confirm availability

extractor = LlmAgent(
    name="extractor",
    model=MODEL,
    instruction="Extract the incident ID and service. Output only verified fields.",
    output_key="extracted_fields",
)

reporter = LlmAgent(
    name="reporter",
    model=MODEL,
    instruction=(
        "Use extracted fields: {extracted_fields}. "
        "Write a cautious summary. Do not invent live evidence."
    ),
)

root_agent = SequentialAgent(
    name="incident_pipeline",
    sub_agents=[extractor, reporter],
)
```

Limitation: this is a control-flow example; the reporter has no verified live incident API in this snippet, so it must not claim to have investigated production telemetry. Add an actual tool step only if necessary.

Graphs: ADK 2.0 mental model

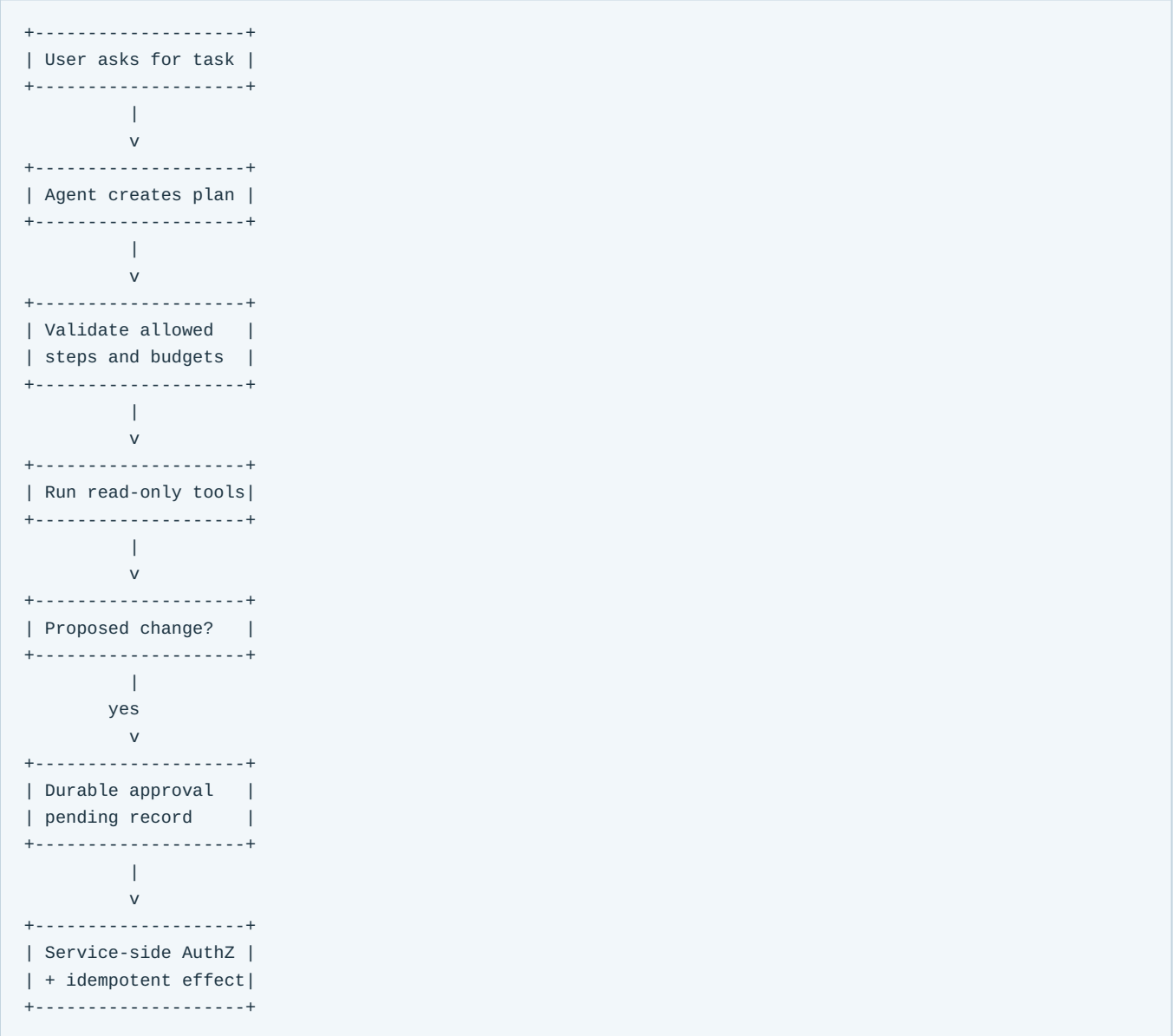
A graph has nodes for function logic, agents, and human input; edges determine transitions. Google documents a `Workflow` with explicit `edges`, including conditional routes [G7a]:

```
# Pseudocode for graph structure, not a complete runnable ADK graph.
# Define and type-check process, route, and result nodes first.
# root_agent = Workflow(
#     name="incident_graph",
#     edges=[
#         ("START", load_incident_node, route_severity_node),
#         (route_severity_node, {
#             "LOW": summary_node,
#             "HIGH": request_review_node,
#         }),
#     ],
# )
```

The graph must define what data each node receives/emits, stop conditions, retries, and how an interrupted run resumes. Do not confuse **a graph drawing** with **durable execution**: persistence and recovery configuration still matter.

9. Bounded planning and human approvals

A common progression:



State transitions for an actual write: `proposed → pending_approval → approved → executing → completed`, with separate `rejected`, `expired`, `failed`, and `unknown` states. An `unknown` outcome means the external provider might have executed the action even when the caller timed out. Reconcile by operation ID before retrying.

Human confirmation in ADK can assist the user experience; Google documents tool confirmations and graph human-input patterns [G6a], [G7b]. But a “Confirm” button, agent message, or resume signal **must not replace** an independent backend authorization check on the frozen request payload.

Example draft record:

```
{
  "operation_id": "restart-INC-42-1",
  "incident_id": "INC-42",
  "service": "payments-api",
  "proposed_action": "restart_service",
  "proposal_hash": "computed-by-server",
  "status": "pending_approval",
  "expires_at": "server-controlled-timestamp",
  "approval_principal": null
}
```

Guardrails: whitelist permissible remediation; require appropriate role; bind reviewer and resource; reject expired/modified proposals; deduplicate retry; record audit and final provider status. Never grant a tool access because the model says “approved.”

---

10. Multi-agent design without unnecessary complexity

Start single-agent. Add specialists only for meaningful separation: distinct contexts, independent tools, ownership, or evaluation. A supervisor plus three specialists is not automatically better than one agent with three tools.

```
+-----------------------------+
| Investigator coordinator    |
+-----------------------------+
               |
       +-------+-------+
       |               |
       v               v
+--------------+ +--------------+
| Metrics      | | Change       |
| specialist   | | specialist   |
+--------------+ +--------------+
       |               |
       +-------+-------+
               |
               v
+-----------------------------+
| Evidence merge + reporting  |
+-----------------------------+
```

Data flow contract for each specialist:

```
{
  "specialist": "metrics",
  "incident_id": "INC-42",
  "finding": "Error rate exceeded mock threshold",
  "evidence_ids": ["health:payments-api:sample-1"],
  "status": "completed"
}
```

A coordinator should detect missing or contradictory evidence and explicitly label unresolved findings. It should not average unsupported model confidence percentages into a fake probability.

A2A is justified when a specialist is an independent remote agent with its own deployment/task lifecycle. For specialists in the same process, ADK sub-agents or plain functions are simpler. ADK has official A2A documentation [G12].

---

11. Context engineering, RAG, and memory

A grounded enterprise agent should answer from authorized and fresh evidence, not accumulated chat history alone.

Retrieval flow: approved documents → ingestion/index → permission-aware search → evidence passages → model synthesis → citation validation. Keep ACL checks before private text enters the model. Record source IDs, versions, and freshness timestamps.

Choice	Use when	Trade-off
Direct context	Small, stable source set	Simple but token heavy
Keyword search	Exact ticket IDs and names	Weak semantic recall
Vector retrieval	Semantic questions over larger corpus	Can miss identifiers
Hybrid + rerank	Both exact and conceptual queries	More components/latency
Long-term memory	User-approved recurring context	Privacy, staleness and consent

Google's ADK context documentation includes session state, memory, context compression, and model context caching [G4], [G5]. Those features help manage context; they are **not a substitute for data access control**.

Failure example: the runbook index contains last month's policy. The model must not present its answer as current without a version check. Solutions include fresh indexing, effective-date metadata, evidence IDs, and abstention when sources conflict.

---

12. Generative UI, MCP Apps, A2A, and ADK

ADK is an **agent runtime/framework**, while MCP, MCP Apps, AG-UI, and A2A address different interfaces. Do not treat them as interchangeable.

Requirement	Recommended boundary	Why
ADK calls shared external tools	MCP client/server	Standardized tool/data integration
Chat host embeds interactive tool UI	MCP Apps *when host supports it*	Sandboxed in-chat HTML view
Your own web UI shows live agent progress	AG-UI or custom server stream	Frontend event/state channel
Independent deployed agent handles a task	A2A	Agent discovery and task/artifact lifecycle
Only need fixed application dashboard	Ordinary typed JSON + HTML	Avoids extra protocols

A realistic ADK deployment can use MCP for tools, A2A for a remote specialist, and a normal web frontend. **MCP Apps is optional** and lives in a compatible host; you do not need it to use ADK or managed Agent Runtime. See [G6], [G12], and [P1]–[P4].

Part III — Managed Vertex AI / Agent Runtime

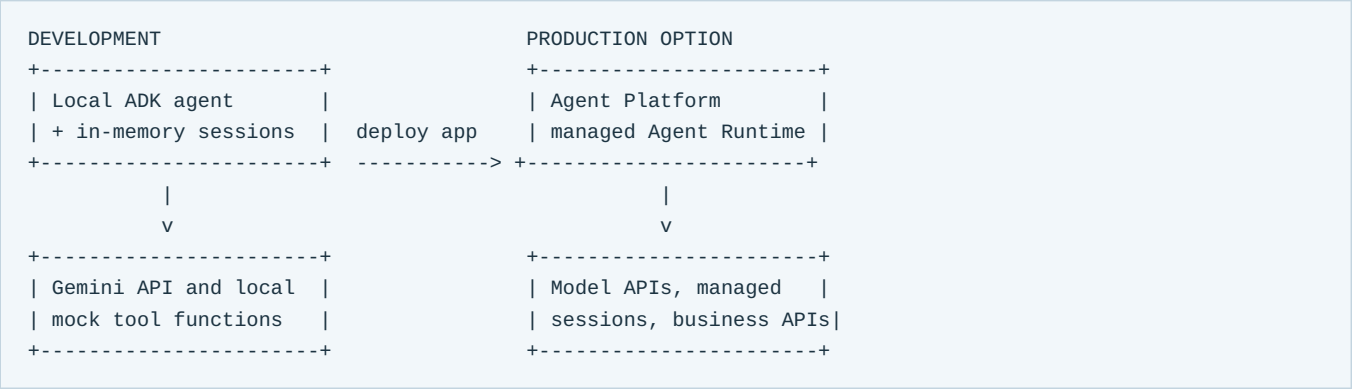
13. Two different meanings of runtime

This distinction prevents many architecture mistakes:

Layer	What it does	Examples
ADK local execution runtime	Executes agent logic and events in an app process	`Runner`, `adk run`, `adk web`
Google Cloud managed Agent Runtime	Hosts deployed agent code behind managed infrastructure	`AdkApp`, `client.agent_engines.create(...)`
Vertex AI / Gemini model API	Generates model outputs	`model="gemini-flash-latest"` in ADK
Session / memory services	Store and retrieve conversation/approved persistent data	ADK or managed services
Business backend	Enforces identity, permissions and write transactions	Ticket, CRM, finance, monitoring APIs

The current Google Cloud quickstart describes the managed host as **Agent Runtime** and deploys a `reasoningEngine` resource using `vertexai` SDK names [G8]. Agent Runtime is part of a broader platform. Do not assume all platform capabilities are enabled, supported in all regions, or included without extra setup.

Local → cloud progression

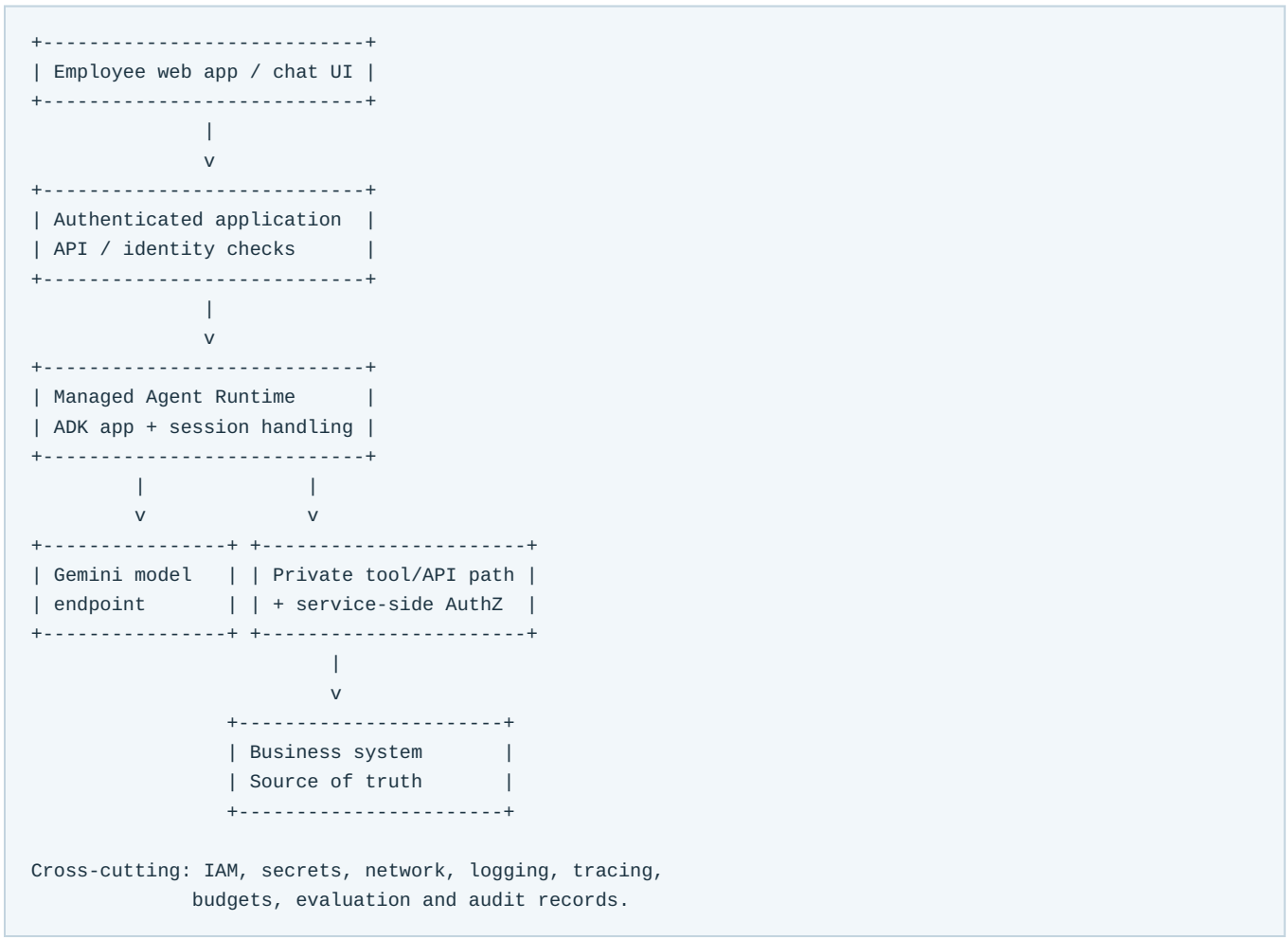


Portability caveat: code using local files, subprocesses, global dictionaries, or local session objects can behave differently once hosted remotely. Evaluate packaging, networking, credentials, dependencies, and state service configuration before deployment.

14. Deployment prerequisites and architecture

The official Google quickstart requires a billable Google Cloud project, enabled APIs, adequate IAM roles, SDK credentials, and a Cloud Storage staging bucket for the example path [G8]. **Do not assume every user can deploy with their default identity.** Grant only the permissions required for building/deploying and runtime execution.

Typical logical production layout:



Trust boundaries: frontend credentials are not agent service credentials; a platform-managed agent identity is not the identity of the human user; authenticated service access still may require per-record authorization; UI presentation is never the approval authority. Inspect current Agent Identity and auth-manager docs for implementation details [G8], [G11].

15. Verified Agent Runtime deployment walkthrough

This walkthrough follows the shape of Google's official managed ADK quickstart [G8]. It provisions billable resources. Run only in a controlled Google Cloud project with permissions, quota, and billing oversight. Some SDK/API names remain historically branded `vertexai`.

15.1 Install and authenticate

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade 'google-cloud-aiplatform[agent_engines,adk]>=1.112'
gcloud auth application-default login
```

Prerequisites to configure outside Python: `PROJECT_ID`, a currently supported `LOCATION`, required Google Cloud APIs/permissions, and `STAGING_BUCKET=gs://YOUR_STAGING_BUCKET`. Verify region support and account policies in current docs. This is **not a zero-cost local command**.

15.2 Wrap and test the ADK agent

```
# deploy_agent.py — based on Google's managed ADK quickstart.
# Run from the project directory and adjust import path as needed.
import os
import vertexai
from vertexai import agent_engines, types
from incident_agent.agent import root_agent

client = vertexai.Client(
    project=os.environ["PROJECT_ID"],
    location=os.environ["LOCATION"],
)

app = agent_engines.AdkApp(agent=root_agent)

# For an optional local test inside an async function:
# async for event in app.async_stream_query(
#     user_id="sample-operator",
#     message="Investigate INC-42",
# ):
#     print(event)
```

15.3 Deploy and query

```
# Continue the same deploy_agent.py after creating client and app.
remote_agent = client.agent_engines.create(
    agent=app,
    config={
        "requirements": ["google-cloud-aiplatform[agent_engines,adk]"],
        "staging_bucket": os.environ["STAGING_BUCKET"],
        "identity_type": types.IdentityType.AGENT_IDENTITY,
    },
)

print("Deployed agent resource:", remote_agent)

# In an async function, you can use:
# async for event in remote_agent.async_stream_query(
#     user_id="sample-operator",
#     message="Investigate INC-42",
# ):
#     print(event)
```

Important deployment note: these are the SDK calls from the current Google quickstart, but the sample `incident_agent` relies on in-process synthetic dictionaries. They are suitable for an isolated demonstration, **not real multi-replica production data**. For production, replace them with authorized remote APIs and configure persistent state.

15.4 Cleanup

Google's quickstart provides `remote_agent.delete(force=True)` for removal of the test agent. Do not run cleanup on a shared/production resource, and review any other associated resources and charges [G8].

15.5 Why staging matters

The staging bucket is an artifact handoff during deployment, not the source of truth for runtime business data. Do not put API keys or customer exports in deployment artifacts. Use approved secrets/identity services for credentials and grant runtime access only to required destinations.

---

16. Managed Agent Runtime vs Cloud Run vs GKE

Google documents all three as deployment options for ADK [G1], [G10]. This is an architectural decision, not a universally correct ranking.

Requirement	Managed Agent Runtime	Cloud Run	GKE
Primary motivation	Agent-focused managed hosting/ integration	Flexible HTTP/container serverless hosting	Kubernetes control and advanced workload customization
Ops burden	Platform-managed runtime; still own agent quality/policy	Own app/container contracts and operations	Own cluster/workload architecture
App packaging	Agent Platform packaging and deployment API	Container/service deployment	Kubernetes workload/image/deployment
Session integration	Review managed session/runtime features	You select session store	You select session store
Networking	Verify supported managed connectivity	Configure Cloud Run networking	Configure Kubernetes networking
Portability	May use platform-specific services	Container application portable with effort	Kubernetes abstractions portable with effort
Good use case	Managed, ADK-native enterprise agents	Custom web API and bespoke agent gateway	Strong platform team and complex enterprise topology

Select a runtime only after identifying identity model, networking, session durability, integration needs, response streaming, region availability, operational controls, budgets, and incident ownership. No deployment choice makes the business data automatically safe.

Part IV — Production-ready design

17. Security and trust boundaries

A production agent must obey the same security principles as any enterprise application: authenticated identity, explicit policy, least privilege, server-side validation, audit, and secure recovery. LLM instructions are not enforcement.

Risk	What can go wrong	Architectural control	Evidence/test
IDOR / cross-record read	Model guesses another user's incident ID	Per-record policy at ticket API	Cross-user read denied
Prompt injection	Retrieved runbook says “ignore previous rules”	Treat source as data, not instructions; restricted tools	Hostile passage cannot trigger write
Insecure write	Generated UI approves service restart	Frozen draft + backend AuthZ + operation ID	Unauthorized action rejected
Duplicate side effect	Timeout followed by retry restarts twice	Idempotency key + reconciliation	One logical action after replay
Secrets leakage	Tool/token appears in traces	Managed secret sources, log redaction, scoped identity	No credentials in model/log output
Tenant leakage	Shared cache/session mixes customers	Tenant isolation in state, caches and APIs	Cross-tenant negative test
Untrusted MCP endpoint	Agent calls malicious advertised server	Approved endpoint registry and auth policy	Unapproved server blocked
Stale evidence	Old runbook used as current policy	Freshness and version checks	Stale source is labelled/ refused

Authentication vs authorization: Google Cloud IAM can decide which service may invoke resources. It does not by itself prove a user can access `INC-42`. The **business API** must still check the caller's actual rights. If the agent runs under a platform identity, design how user identity/entitlement is propagated or rechecked.

Recommended controlled action sequence: propose → persist immutable draft → explicit approval → backend policy decision → idempotent business operation → audit/reconciliation. Never ask the model to determine whether it has permission to do a write.

Google's ADK docs cover tool authentication/confirmations and Cloud Agent Identity. For broader principles see [G6a], [G11], [S1], [S2].

18. Evaluation, testing, and observability

A service can have 99.9% HTTP uptime and still give wrong answers. **Agent quality is not service availability.** Evaluate both.

Evaluation layers

Layer	Example automated check	Success criterion
Tool unit test	`get_incident("INC-999")`	Returns `not_found`, no exception
Auth test	Operator A reads operator B's private record	`forbidden` before model sees data
Integration	Incident API times out	Clear error event, no invented health result
Trajectory	Question about one ticket chooses intended tool	Correct tool and parameters
Grounding	Incident report cites evidence IDs	Every cited ID exists in tool result
Task outcome	Investigation report covers known test scenario	Meets rubric and policy
Nonfunctional	Model/tool latency and token usage	Within chosen SLO and budget
Reliability	Retry after ambiguous write	No duplicate logical effect

Google ADK documents agent evaluation criteria and trace/metric instrumentation. Current Agents CLI documentation also covers dataset-driven `eval run`, `eval grade`, `eval compare`, and `eval analyze` workflows. Pin a tool version and inspect its current dataset schema before using generated eval files [G13], [G14].

Observability fields: request ID, run ID, session ID (redacted/hashed as necessary), user/tenant pseudonymous identifier, agent version, model, tool name, tool latency, token usage, endpoint, error class, evidence IDs, and final task outcome. Do not log raw secrets or entire sensitive prompts by default.

Useful metrics: task-success rate, supported-claim rate, human-correction rate, tool errors, p50/p95 user-visible latency, timeout rate, cost per successful task, unauthorized requests blocked, and stale-evidence rate. Instrument each hop so a reviewer can explain a failure timeline.

Release management

1. Freeze a version of agent instructions, model choice, tools and dependencies.
2. Run golden evaluation scenarios and negative authorization tests.
3. Deploy candidate to a non-production environment.
4. Use a controlled canary/traffic split when supported.
5. Roll back on task-quality, security, cost or latency regression.
6. Treat schema/state migration separately from prompt-only updates.

19. Reliability, scalability, and cost design

Agent systems fail in more ways than standard REST servers: model refusal/error, malformed tool arguments, upstream timeout, duplicate writes, stale context, infinite loops, token costs, and disconnected clients.

Failure/recovery matrix

Failure	Detect	Safe response	Design change
Model cannot choose a tool	Step limit/trace	Ask for clarification or return bounded failure	Simplify tool catalog; evaluate routing
Tool returns `forbidden`	Service error code	Do not retry with broader privileges	Validate authorization path
Ticket API times out	Timeout budget	Label upstream unavailable, no invented status	Bounded retries/circuit breaker
Run crashes before summary	Missing completed checkpoint	Resume from persisted state or restart read-only task	Durable state/checkpoints
Write times out after provider accepted it	Operation state `unknown`	Reconcile provider by operation ID	Idempotency ledger
Prompt injection in retrieved text	Policy/review signal	Ignore instructions in source content	Sandbox, tool constraints
UI stream disconnects	Client sequence gap	Show reconnecting/unknown status	Reconnect with event cursor/ snapshot
Traffic spike	Queue depth and throttles	Apply admission control and fair limits	Concurrency, quotas, budgets

Performance and budget levers

- Keep tool catalogs small and task-relevant; large schemas consume context.
- Prefer deterministic code for routing where possible.
- Run independent read-only retrieval in parallel when it meaningfully reduces total latency.
- Keep tool timeouts shorter than the total user-request budget.
- Cache only authorized, versioned, non-sensitive answers when appropriate.
- Measure cost **per successful task**, not just cost per model call.
- Keep tokens bounded: concise tool results, summarized history, evidence selection, explicit step/loop limits.
- A more capable model may reduce retries; benchmark total task outcome instead of comparing input-token price alone.
- For long-running work, use a durable job/approval mechanism rather than an unbounded browser request.

Do not invent cloud SLAs, quotas, region availability, or prices. They are version-, region-, and account-specific. Confirm current figures in Google Cloud's official pricing/limits pages before committing an architecture.

Part V — Use-case blueprints

20. Eight real-world ADK solution architectures

UC1 — IT incident investigator (foundation → advanced)

Business goal: summarize an incident using monitoring and tickets. **Initial design:** one ADK agent and two read-only tools; no persistent memory or multi-agent system. **Advanced design:** add a workflow for evidence ordering, optional MCP monitoring adapter, approved runbooks, evaluation and external audit. **Source of truth:** ticket/monitoring systems. **Risks:** stale evidence, hallucinated root cause, unauthorized remediation. **Deliverable:** cited report with `unverified` and `inconclusive` states.

UC2 — Customer support / order status

Business goal: answer a customer's order status without leaking another customer's details. **Initial design:** deterministic identity/ownership check plus a read-only order tool. **Advanced design:** RAG for approved shipping policy, isolated sessions, customer-intent routing. **Do not:** let an LLM-provided `order_id` bypass order-owner authorization. **Architecture choice:** a plain API/LLM combination may be better than a large workflow.

UC3 — Enterprise document Q&A

Business goal: answer questions from permissioned policy documents. **Initial design:** retrieval with source IDs + one synthesizing agent. **Advanced design:** tenant-aware hybrid search, version/freshness enforcement, citation validation, corpus evaluation. **Source of truth:** document management system; search index is derived. **Risks:** outdated policy, forbidden text in model context, fake citations.

UC4 — Refund or expense approval assistant

Business goal: propose a financial adjustment and obtain human approval. **Initial design:** an agent drafts; deterministic service enforces policy. **Advanced design:** durable graph, immutable approval payload, reviewer role and expiry, idempotency ledger and unknown-outcome reconciliation. **Never:** let a generated approval button be the final authority. **Good fit:** ADK agent + workflow API/business state machine.

UC5 — Supplier onboarding workflow

Business goal: coordinate document collection, compliance review and approvals over days. **Initial design:** deterministic status tracker plus agent-generated summaries. **Advanced design:** human-input graph, durable checkpoints, asynchronous callbacks, cancellation, audit, event-triggered resumption. **Risk:** lost progress when an agent process restarts. **Good fit:** workflow engine plus ADK for bounded reasoning steps.

UC6 — Research and report generator

Business goal: retrieve evidence, compare sources, and produce a cited report. **Initial design:** fixed retrieve → summarize pipeline. **Advanced design:** parallel evidence gathering, independent claim verification, source freshness, budgets and evaluation. **Risk:** two agents agreeing on the same unsupported fact. **Good fit:** ADK graph/sequential or parallel template, not unrestricted free-form research loops.

UC7 — Multi-agent SOC / fraud-review coordination

Business goal: coordinate an internal case with a separately operated specialist. **Initial design:** in-process specialist function or sub-agent. **Advanced design:** external specialist via A2A, signed/trusted endpoint policy, task ID, status tracking, artifact schema validation. **Risk:** spoofed agent capabilities, disclosure of sensitive case information. **Good fit:** A2A only across independently deployed service boundaries.

UC8 — Agent-powered operations dashboard

Business goal: present live progress, evidence, and an interactive reviewer queue. **Initial design:** ADK backend returns typed JSON to plain HTML. **Advanced design:** frontend streaming using AG-UI or a supported event channel; host-embedded MCP App if using a compatible conversational host. **Risk:** frontend showing action completed when backend status is unknown. **Good fit:** regular HTML UI + explicit state protocol; MCP Apps only for host embedding.

Which use case should drive the capstone?

Use **UC1 IT incident investigator** as the central project because it progresses naturally from one safe read-only agent to a full managed Agent Runtime solution. Add UC4's approval workflow only after the initial read-only design is reliable. Add MCP/A2A/GenUI only at the point where a real integration requirement demands them.

Part VI — Hands-on path and design decisions

21. Progressive project labs (software tests, not learner quizzes)

Each lab has visible outputs and meaningful failure behavior. Use only synthetic records.

Lab	Build	Input	Observable result	Important negative test
1	Local single ADK agent	`Investigate INC-42`	Answer cites mock incident and health IDs	`INC-999` reports not found
2	Programmatic Runner/ session	Two turns about same incident	Session events/state visible	New session has no prior private context
3	Typed tool wrapper	Valid incident ID	Schema-valid result with error status	Malformed ID rejected
4	Deterministic sequential workflow	Evidence + report request	Fixed order and bounded steps	Missing evidence prevents unsupported claim
5	ADK 2.0 graph with human-input step	Proposed mock restart	Durable `pending_approval` state	Modified/expired approval rejected
6	MCP tool integration	Authorized remote ticket query	Typed tool call across client/ server boundary	Unauthorized record denied by server
7	Agent Runtime deployment	Managed run of incident tool example	Remote events and resource ID	Wrong IAM role / missing bucket fails clearly
8	Production evaluation/ trace	Golden incident set	Tool + task results traceable	Injected runbook text cannot trigger write

Minimal deterministic mock test file: place this alongside the agent package and use Python standard library for the function tests.

```
# test_mock_tools.py — run: python test_mock_tools.py
from incident_agent.agent import get_incident, get_service_health

assert get_incident("INC-42")["status"] == "success"
assert get_incident("INC-999")["status"] == "not_found"
assert get_service_health("payments-api")["health"]["status"] == "degraded"
assert get_service_health("missing-service")["status"] == "not_found"
print("Mock tool checks passed")
```

Practical sequence: run Labs 1–4 without cloud deployment. For Lab 5, use the official ADK 2.0 human-input and graph API documentation. For Lab 6, use a separate synthetic MCP server with real server-side ACL tests. For Lab 7, isolate charges in a test project. Lab 8 must check both positive task outcomes and negative security paths.

---

22. Production architecture package / ADR template

Use the following headings for every professional architecture review:

1. **Problem and source of truth:** who requests the outcome; what business systems own the facts.
2. **Requirements:** capabilities, excluded scope and ranked qualities.
3. **Proposed boundary diagram:** who hosts the agent; which calls cross services.
4. **Agent/tool contracts:** typed inputs, results, expected errors.
5. **Identity and authorization:** user → app → agent → data/tool path.
6. **State and durability:** session owner, workflow record, checkpoint/recovery strategy.
7. **Failure behavior:** timeout, lost response, retries, cancellation, `unknown` outcomes.
8. **Observability and evaluation:** trace IDs, success metrics, negative tests.
9. **Operations:** environments, IAM, secrets, rollout, rollback, budgets, cleanup.
10. **Alternatives and trade-offs:** why ADK versus plain API; why managed runtime versus Cloud Run; whether MCP/A2A adds value.

Example architecture decision record

Decision: Host a read-only incident investigation agent in managed Agent Runtime when the organization needs platform-managed deployment and integration. **Alternative:** Cloud Run with the ADK app packaged as a service. **Reason:** choose based on runtime integration and operational requirements rather than the ADK framework itself. **Trade-offs:** platform-specific deployment coupling, IAM and network setup, billable resources, service compatibility. **Revisit if:** custom server lifecycle or networking constraints make Cloud Run/GKE a better fit.

---

23. Final decision checklist

Use a single agent when: tool set is small, the problem is one bounded reasoning task, one team owns all APIs, and there is no need to resume a complex workflow.

Use an explicit ADK workflow when: steps must execute in order, approvals are required, failures must resume predictably, or parallel analysis needs controlled aggregation. For ADK 2.0, review graph workflow APIs first.

Use managed Agent Runtime when: managed hosting is a justified organizational requirement and its current connectivity, security, region and runtime contract meet your needs.

Use Cloud Run/GKE instead when: custom HTTP/service control, container operations, platform constraints, or environment policies make general-purpose hosting a better fit.

Use MCP when: tool interoperability across hosts/agents matters; not automatically for every Python function. **Use A2A when:** independent remote agents require task contracts. **Use MCP Apps when:** a compatible AI host must display an embedded interactive view. **Use AG-UI when:** an owned frontend needs a standardized agent event stream.

Before any high-impact operation: persist proposal, enforce policy outside the model, require valid authorization, assign stable operation ID, check current business state, execute idempotently, and reconcile unknown outcomes.

Glossary

- **ADK:** Google Agent Development Kit, an agent development/orchestration framework.
- **Agent:** application object combining model behavior, instructions and tools.
- **A2A:** Agent2Agent protocol for independently operated agent systems.
- **Agent Runtime:** managed Google Cloud host for agent applications (current platform naming).
- **Artifact:** non-chat output such as a report or file.
- **Callback:** lifecycle hook for observing/customizing/controlling supported agent stages.
- **Context:** information supplied to the model for the current generation step.
- **Event:** execution/response/state record emitted by agent runtime.
- **Idempotency:** a retry of the same logical operation does not duplicate its effect.
- **MCP:** Model Context Protocol for host/client ↔ tool/data server integration.
- **MCP Apps:** optional extension for host-embedded interactive HTML views.
- **Memory:** authorized reusable information beyond a single turn/session.
- **OpenAPI tool:** tool generated or configured from REST API description.
- **Runner:** component invoking ADK agent execution and yielding events.
- **Session:** persistent or temporary conversation/agent state scope.
- **Source of truth:** authoritative business system for a fact/transaction.
- **State:** workflow/session values used across steps.
- **Tool:** callable function/service available to an agent within bounds.
- **Vertex AI Agent Engine:** earlier name for the managed agent hosting capability now documented as Agent Runtime in Gemini Enterprise Agent Platform.
- **Workflow graph:** explicit nodes/edges that constrain execution order and transitions.

Official source library

Google and Google Cloud (primary references)

- **[G1]** Google ADK documentation and framework overview — https://adk.dev/

- **[G2]** ADK Python quickstart (`pip install` , `adk create` , `adk run` , `adk web`) — https://adk.dev/get-started/python/
- **[G3]** ADK event loop / local execution — https://adk.dev/runtime/event-loop/
- **[G4]** ADK state scopes — https://adk.dev/sessions/state/
- **[G5]** ADK memory — https://adk.dev/sessions/memory/
- **[G6]** MCP tools with ADK — https://adk.dev/tools-custom/mcp-tools/
- **[G6a]** Advanced MCP configuration (auth, confirmations) — https://adk.dev/tools-custom/mcp-tools/advanced/
- **[G7]** Sequential workflow agents — https://adk.dev/agents/workflow-agents/sequential-agents/
- **[G7a]** ADK 2.0 graph routes — https://adk.dev/graphs/routes/
- **[G7b]** ADK graph human input — https://adk.dev/graphs/human-input/
- **[G8]** Google Cloud: develop and deploy ADK to Agent Runtime — https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/runtime/quickstart-adk
- **[G9]** ADK deploy to Agent Runtime — https://adk.dev/deploy/agent-engine/
- **[G10]** Google Cloud architecture: ADK on Cloud Run — https://docs.cloud.google.com/architecture/single-agent-ai-system-adk-cloud-run
- **[G11]** Gemini Enterprise Agent Platform overview / cloud documentation — https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk
- **[G12]** ADK A2A introduction — https://adk.dev/a2a/intro/
- **[G13]** ADK evaluation — https://adk.dev/evaluate/
- **[G14]** Google Agents CLI evaluation guide — https://google.github.io/agents-cli/guide/evaluation/
- **[G15]** ADK callbacks — https://adk.dev/callbacks/
- **[G16]** ADK metrics — https://adk.dev/observability/metrics/

Protocol, security and operational references

- **[P1]** MCP specification — https://modelcontextprotocol.io/specification
- **[P2]** MCP Apps — https://github.com/modelcontextprotocol/ext-apps
- **[P3]** AG-UI protocol — https://github.com/ag-ui-protocol/ag-ui
- **[P4]** A2A protocol — https://a2a-protocol.org/
- **[S1]** OWASP GenAI security project — https://genai.owasp.org/
- **[S2]** OpenTelemetry GenAI conventions — https://opentelemetry.io/docs/specs/semconv/gen-ai/

Version practice: pin ADK, Python and Google Cloud SDK versions for a project; revisit the relevant official page when a class, package, model identifier, IAM role, region or service name changes. No pricing, quota or SLA figure is asserted in this guide.